

Java

Java 速成

关于 Java

Java 是一种广泛使用的计算机编程语言，拥有 **跨平台**、**面向对象**、**泛型编程** 的特性，广泛应用于企业级 Web 应用开发和移动应用开发。

环境安装

参见 [JDK](#)。

基本语法

主函数

Java 类似 C/C++ 语言，需要一个函数（在面向对象中，这被称为方法）作为程序执行的入口点。

Java 的主函数的格式是固定的，形如：

```
1 class Test {
2     public static void main(String[] args) {
3         // 程序的代码
4     }
5 }
```

一个打包的 Java 程序（名称一般是 *.jar）中可以有很多个类似的函数，但是当运行这个程序的时候，只有其中一个函数会被运行，这是定义在 Jar 的 Manifest 文件中的，在 OI 比赛中一般用不到关于它的知识。

注释

和 C/C++ 一样，Java 使用 // 和 /* */ 分别注释单行和多行。

基本数据类型

类型名	意义
boolean	布尔类型
byte	字节类型
char	字符型
double	双精度浮点
float	单精度浮点
int	整型
long	长整型
short	短整型
null	空

声明变量

```
1 int a = 12; // 设置 a 为整数类型, 并给 a 赋值为 12
2 String str = "Hello, OI-wiki"; // 声明字符串变量 str
3 char ch = 'W';
4 double PI = 3.1415926;
```

final 关键字

`final` 含义是这是最终的、不可更改的结果, 被 `final` 修饰的变量只能被赋值一次, 赋值后不再改变。

```
1 final double PI = 3.1415926;
```

数组

```
1 // 有十个元素的整数类型数组
2 // 其语法格式为 数据类型[] 变量名 = new 数据类型[数组大小]
3 int[] ary = new int[10];
```

字符串

- 字符串是 Java 一个内置的类。

```
1 // 最为简单的构造一个字符串变量的方法如下
2 String a = "Hello";
3
4 // 还可以使用字符数组构造一个字符串变量
5 char[] stringArray = { 'H', 'e', 'l', 'l', 'o' };
6 String s = new String(stringArray);
```

包和导入包

Java 中的类 (Class) 都被放在一个个包 (package) 里面。在一个包里面不允许有同名的类。在类的第一行通常要说明这个类是属于哪个包的。例如：

```
1 package org.oj-wiki.tutorial;
```

包的命名规范一般是：项目所有者的顶级域.项目所有者的二级域.项目名称。

通过 `import` 关键字来导入不在本类所属的包下面的类。例如下面要用到的 `Scanner`：

```
1 import java.util.Scanner;
```

如果想要导入某包下面所有的类，只需要把这个语句最后的分号前的类名换成 `*`。

输入

可以通过 `Scanner` 类来处理命令行输入。

```

1 package org.oiwiki.tutorial;
2
3 import java.util.Scanner;
4
5 class Test {
6     public static void main(String[] args) {
7         Scanner scan = new Scanner(System.in); // System.in 是输入流
8         int a = scan.nextInt();
9         double b = scan.nextDouble();
10        String c = scan.nextLine();
11    }
12 }

```

输出

可以对变量进行格式化输出。

符号 意义

%f 浮点类型
 %s 字符串类型
 %d 整数类型
 %c 字符类型

```

1 class Test {
2     public static void main(String[] args) {
3         int a = 12;
4         char b = 'A';
5         double s = 3.14;
6         String str = "Hello world";
7         System.out.printf("%f\n", s);
8         System.out.printf("%d\n", a);
9         System.out.printf("%c\n", b);
10        System.out.printf("%s\n", str);
11    }
12 }

```

控制语句

Java 的流程控制语句与 C++ 是基本相同的。

选择

- if

```

1 class Test {
2     public static void main(String[] args) {
3         if ( /* 判断条件 */ ){
4             // 条件成立时执行这里的代码
5         }
6     }
7 }

```

- if...else

```

1 class Test {
2     public static void main(String[] args) {
3         if ( /* 判断条件 */ ) {
4             // 条件成立时执行这里的代码
5         } else {
6             // 条件不成立时执行这里的代码
7         }
8     }
9 }

```

- if...else if...else

```

1 class Test {
2     public static void main(String[] args) {
3         if ( /* 判断条件 */ ) {
4             //判断条件成立执行这里的代码
5         } else if ( /* 判断条件2 */ ) {
6             // 判断条件2成立执行这里的代码
7         } else {
8             // 上述条件都不成立执行这里的代码
9         }
10    }
11 }

```

- switch...case

```

1 class Test {
2     public static void main(String[] args) {
3         switch ( /* 表达式 */ ){
4             case /* 值 1 */:
5                 // 当表达式取得的值符合值 1 执行此段代码
6                 break; // 如果不加上 break 语句,会让程序按顺序往下执行直到 break
7             case /* 值 2 */:
8                 // 当表达式取得的值符合值 2 执行此段代码
9                 break;
10            default:
11                // 当表达式不符合上面列举的值的时候执行这里的代码
12        }
13    }
14 }

```

循环

- for

for 关键字有两种使用方法，其中第一种是普通的 for 循环，形式如下：

```

1 class Test {
2     public static void main(String[] args) {
3         for ( /* 初始化 */; /* 循环的判断条件 */; /* 每次循环后执行的步骤 */ ) {
4             // 当循环的条件成立执行循环体内代码
5         }
6     }
7 }

```

第二种是类似 C++ 的 foreach 使用方法，用于循环数组或者集合中的数据，相当于把上一种方式中的循环变量隐藏起来了，形式如下：

```

1 class Test {
2     public static void main(String[] args) {
3         for ( /* 元素类型X */ /* 元素名Y */ : /* 集合Z */ ) {
4             // 这个语句块的每一次循环时，元素Y分别是集合Z中的一个元素。
5         }
6     }
7 }

```

- while

```

1 class Test {
2     public static void main(String[] args) {
3         while ( /* 判定条件 */ ) {
4             // 条件成立时执行循环体内代码
5         }
6     }
7 }

```

- do...while

```

1 class Test {
2     public static void main(String[] args) {
3         do {
4             // 需要执行的代码
5         } while ( /* 循环判断条件 */ );
6     }
7 }

```

注意事项

类名与文件名一致

创建 Java 源程序需要类名和文件名一致才能编译通过，否则编译器会提示找不到类。通常该文件名会在具体 OJ 中指定。

例：

Add.java

```

1 class Add {
2     public static void main(String[] args) {
3         // ...
4     }
5 }

```

在该文件中需使用 Add 为类名方可编译通过。

本页面最近更新：2023/7/24 00:21:30, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[qyl27](#), [1804040636](#), [billchenchina](#), [clansty](#), [CoelacanthusHex](#), [countercurrent-time](#), [Cubik65536](#), [diauweb](#), [Enter-tainer](#), [gtn1024](#), [H-J-Granger](#), [Henry-ZHR](#), [lr1d](#), [jingyuexing](#), [Konano](#), [ksyx](#), [mgt](#), [NachtgeistW](#), [orzAtalod](#), [shuzhouliu](#), [SukkaW](#), [Suyun514](#), [Tiphereth-A](#), [Xeonacid](#), [yusancky](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用